

Algorithms and Complexity for Some Private Neighbor Parameters

Zhilan Wang*

Shuo Wei†

Abstract

The notion of a private neighbor is a fundamental local certificate in domination and irredundance theory. Motivated by Hedetniemi and Rall’s recent preprint [arXiv:2511.07248], this paper addresses two questions raised there: how to compute the resulting private neighbor parameters on trees, and what their computational complexity is on general graphs. First, we give a unified dynamic programming algorithm that computes all six parameters on trees in linear time and also constructs an optimal set. Second, we prove that, on arbitrary finite simple graphs, the decision problems are NP-complete by reductions from INDEPENDENT SET and MAX-CUT. Consequently, the corresponding exact optimization problems are NP-hard on general graphs.

Keywords: Private neighbors; trees; dynamic programming; NP-completeness; Max-Cut

Mathematics Subject Classifications: 05C69; 05C85; 68Q25

1 Introduction

Domination and irredundance are two closely related themes in graph theory. A set D of vertices in a graph G is a dominating set if every vertex outside D has a neighbor in D . The domination number and its many variants have been studied extensively from structural, extremal and algorithmic points of view; see, for example, the monograph of Haynes, Hedetniemi and Henning [9]. A central local notion in this area is that of a private neighbor. This concept was introduced by Cockayne, Hedetniemi and Miller [4] and has since become a standard tool in the study of domination, irredundance and related graph parameters.

Let $G = (V, E)$ be a finite simple graph and let $U \subseteq V(G)$. For $x \in V(G)$, write $p_U(x) = |N_G(x) \cap U|$. A vertex $x \notin U$ with $p_U(x) = 1$ is called an *external private neighbor* with respect to U . A vertex $x \in U$ with $p_U(x) = 1$ is called an *internal private neighbor* with respect to U . Finally, a vertex $x \in U$ with $p_U(x) = 0$ is called a *self private neighbor* with respect to U . These notions unify several classical objects. For instance, independent sets are precisely those sets all of whose vertices are self private neighbors, while open irredundant sets are closely related to sets whose vertices have external private neighbors. Irredundance and its variants have been studied in many forms; see, for

*School of Mathematics, Shandong University, Jinan 250100, China. Corresponding author. Email: zhilanwang@mail.sdu.edu.cn. Supported by the Chongqing Key Project (No. KJZD-K202502601).

†School of Mathematics, Shandong University, Jinan 250100, China.

example, [1, 3, 6, 10, 13]. Related parameters include induced matchings [2], 1-dependence [7], and open irredundance [5].

Recently, Hedetniemi and Rall [11] introduced a family of parameters obtained by maximizing the number of vertices that occur as private neighbors of a suitable set. For a set $U \subseteq V(G)$, let $E(U)$, $I(U)$ and $S(U)$ denote the numbers of external, internal and self private neighbors with respect to U , respectively; the formal definitions are given in Section 2. The six parameters studied in this paper are

$$\begin{aligned} IPN(G) &= \max I(U), & EPN(G) &= \max E(U), \\ ESPN(G) &= \max(E(U) + S(U)), & EIPN(G) &= \max(E(U) + I(U)), \\ ISPN(G) &= \max(I(U) + S(U)), & EISPN(G) &= \max(E(U) + I(U) + S(U)), \end{aligned} \tag{1.1}$$

where in each case the maximum is taken over all $U \subseteq V(G)$. These parameters measure how many vertices can be forced into prescribed private neighbor roles. Thus the emphasis is not on maximizing the certificate set U itself, but on maximizing the number of vertices certified by U .

Hedetniemi and Rall [11] determined exact values for several basic graph classes, including paths, cycles and complete bipartite graphs, and posed the following two problems.

Problem 1.1 (Hedetniemi–Rall [11]). *Construct algorithms to compute each of $IPN(T)$, $EPN(T)$, $ESPN(T)$, $EIPN(T)$, $ISPN(T)$ and $EISPN(T)$ for a tree T .*

Problem 1.2 (Hedetniemi–Rall [11]). *Determine the complexity of computing each of $IPN(G)$, $EPN(G)$, $ESPN(G)$, $EIPN(G)$, $ISPN(G)$ and $EISPN(G)$ for a graph G .*

In this paper we resolve both problems. Our first result gives a unified linear-time algorithm on trees.

Theorem 1.3. *For every tree T , the six values*

$$IPN(T), \quad EPN(T), \quad ESPN(T), \quad EIPN(T), \quad ISPN(T), \quad EISPN(T)$$

can be computed in linear time. Moreover, for each of the six parameters, a corresponding optimal set $U \subseteq V(T)$ can also be constructed in linear time.

Our second result shows that this efficient behavior on trees does not extend to arbitrary graphs. For a parameter $p \in \{IPN, EPN, ESPN, EIPN, ISPN, EISPN\}$, let p -PRIVATE-NEIGHBOR denote the following decision problem: given a graph G and an integer k , decide whether $p(G) \geq k$.

Theorem 1.4. *For each*

$$p \in \{IPN, EPN, ESPN, EIPN, ISPN, EISPN\},$$

the problem p -PRIVATE-NEIGHBOR is NP-complete on arbitrary finite simple graphs.

The rest of this paper is organized as follows. In Section 2 we give the necessary definitions and notation. In Section 3 we present the unified dynamic programming algorithm on trees and prove Theorem 1.3. In Section 4 we prove Theorem 1.4. More precisely, in Subsection 4.1 we prove the NP-completeness results for *IPN* and *ISPN*, and in Subsection 4.2 we prove the corresponding results for *EPN*, *EIPN*, *ESPN* and *EISPN*. The appendix records pseudocode for the tree dynamic program.

2 Preliminaries

All graphs considered in this paper are finite, simple and undirected. For a graph G and a vertex $v \in V(G)$, we write $N_G(v)$ and $N_G[v]$ for the open and closed neighborhoods of v , respectively. When the graph is clear from the context, we omit the subscript G .

Let $U \subseteq V(G)$, and put $p_U(x) = |N_G(x) \cap U|$ for each $x \in V(G)$. A vertex x is an external private neighbor with respect to U if $x \notin U$ and $p_U(x) = 1$. A vertex x is an internal private neighbor with respect to U if $x \in U$ and $p_U(x) = 1$, or equivalently if x has degree 1 in $G[U]$. A vertex x is a self private neighbor with respect to U if $x \in U$ and $p_U(x) = 0$, or equivalently if x is isolated in $G[U]$.

We denote by $E(U)$, $I(U)$ and $S(U)$ the numbers of external, internal and self private neighbors with respect to U , respectively. With this notation, the six parameters studied in this paper are those defined in (1.1).

3 A linear-time algorithm for trees

We prove Theorem 1.3 in this section. The proof is based on a unified dynamic program which computes all six parameters by changing only three fixed coefficients.

We first give a unified formulation which covers all six parameters. Fix $\alpha, \beta, \gamma \in \{0, 1\}$ and, for $U \subseteq V(G)$, set

$$\Phi_{\alpha, \beta, \gamma}(G, U) = \alpha E(U) + \beta I(U) + \gamma S(U).$$

Thus the six parameters correspond to the following choices of (α, β, γ) :

Table 1: Coefficient choices for the six parameters.

Parameter	(α, β, γ)
<i>IPN</i>	(0, 1, 0)
<i>EPN</i>	(1, 0, 0)
<i>ESPN</i>	(1, 0, 1)
<i>EIPN</i>	(1, 1, 0)
<i>ISPN</i>	(0, 1, 1)
<i>EISPN</i>	(1, 1, 1)

We also define a local contribution function. For $b \in \{0, 1\}$ and an integer $t \geq 0$, let

$$\varphi_{\alpha, \beta, \gamma}(b, t) = \begin{cases} \alpha, & b = 0 \text{ and } t = 1, \\ \beta, & b = 1 \text{ and } t = 1, \\ \gamma, & b = 1 \text{ and } t = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Here b records whether the vertex itself belongs to U , while t records the number of its neighbours that belong to U .

Let T be a non-empty tree, rooted at an arbitrary vertex r . For a vertex v , let $C(v)$ be the set of children of v , and let T_v be the subtree rooted at v . If $v \neq r$, let $p(v)$ denote the parent of v .

For $a, b \in \{0, 1\}$, define $F_v(a, b)$ to be the maximum weighted value contributed by all vertices of T_v , under the following boundary conditions: $v \in U$ if and only if $b = 1$; if $v \neq r$, then the parent $p(v)$ is regarded as belonging to U if and only if $a = 1$; and every vertex of T_v is evaluated as it would be in the whole tree, with the parent of v treated according to a . This boundary information is sufficient because the only possible edge from T_v to the rest of the tree is the edge from v to its parent.

For fixed v and $b \in \{0, 1\}$, define auxiliary values $G_v^b(q)$ for $q \in \{0, 1, 2\}$. The value $G_v^b(q)$ is the maximum total contribution from the subtrees rooted at the children of v , over all choices of U inside these subtrees, subject to

$$q = \min\{2, |\{u \in C(v) : u \in U\}|\}.$$

Here q records the number of children of v that belong to U , capped at 2.

These values are computed by processing the children one by one. Start with

$$H_0(0) = 0, \quad H_0(1) = H_0(2) = -\infty.$$

Suppose the first s children have been processed and the current values are $H_s(0), H_s(1), H_s(2)$. If the next child is u , define, for $q' \in \{0, 1, 2\}$,

$$H_{s+1}(q') = \max \{H_s(q) + F_u(b, c) : q \in \{0, 1, 2\}, c \in \{0, 1\}, q' = \min\{2, q + c\}\}.$$

After all children have been processed, set

$$G_v^b(q) := H_{|C(v)|}(q) \quad (q = 0, 1, 2).$$

If v is a leaf, then $G_v^b(0) = 0$ and $G_v^b(1) = G_v^b(2) = -\infty$.

The recurrence for $F_v(a, b)$ is

$$F_v(a, b) = \max_{q \in \{0, 1, 2\}} \left(G_v^b(q) + \varphi_{\alpha, \beta, \gamma}(b, a + q) \right),$$

where $\varphi_{\alpha, \beta, \gamma}(b, a + q)$ is interpreted using the integer $a + q$; if $a + q \geq 2$, then the local contribution is automatically 0. At the root there is no parent, so the desired value is

$$\max_{U \subseteq V(T)} \Phi_{\alpha, \beta, \gamma}(T, U) = \max\{F_r(0, 0), F_r(0, 1)\}.$$

Proof of Theorem 1.3. We prove the following slightly more general assertion: for every tree T and every $(\alpha, \beta, \gamma) \in \{0, 1\}^3$, the dynamic program described above computes

$$\max_{U \subseteq V(T)} \Phi_{\alpha, \beta, \gamma}(T, U)$$

in linear time and constructs an optimal set U in linear time.

We prove the recurrence by induction on the rooted subtree T_v . Fix v and $a, b \in \{0, 1\}$. Suppose first that the membership of every child $u \in C(v)$ has been chosen, and write $c_u = 1$ if $u \in U$, and $c_u = 0$ otherwise. Once b and c_u are fixed, the optimization inside the subtree T_u is independent of all other child-subtrees, because the only edge from T_u to the rest of the tree is uv . Thus the best possible contribution from T_u is exactly $F_u(b, c_u)$.

The contribution of v itself depends only on whether $v \in U$, whether its parent belongs to U , and how many of its children belong to U . Under the boundary condition, these data are precisely b , a , and the number $\sum_{u \in C(v)} c_u$. Hence the contribution of v is

$$\varphi_{\alpha, \beta, \gamma} \left(b, a + \sum_{u \in C(v)} c_u \right).$$

Since $\varphi_{\alpha, \beta, \gamma}$ distinguishes only the cases where the number of selected neighbors is 0, 1, or at least 2, the capped value

$$q = \min \left\{ 2, \sum_{u \in C(v)} c_u \right\}$$

contains all information needed to evaluate the contribution of v .

Therefore, for fixed a and b , the optimal value is obtained by maximizing

$$\sum_{u \in C(v)} F_u(b, c_u) + \varphi_{\alpha, \beta, \gamma} \left(b, a + \sum_{u \in C(v)} c_u \right)$$

over all choices $c_u \in \{0, 1\}$. The auxiliary dynamic program defining $G_v^b(q)$ performs exactly this maximization over the child-subtrees while retaining the capped number q of selected children. Thus

$$F_v(a, b) = \max_{q \in \{0, 1, 2\}} \left(G_v^b(q) + \varphi_{\alpha, \beta, \gamma}(b, a + q) \right).$$

For a leaf v , there are no child-subtrees, and the recurrence reduces to

$$F_v(a, b) = \varphi_{\alpha, \beta, \gamma}(b, a),$$

which is exactly the correct contribution of the single vertex v under the prescribed parent membership. Induction from the leaves to the root proves that all values $F_v(a, b)$ are correct. At the root, the absence of a parent is represented by $a = 0$, and maximizing over whether the root belongs to U gives

$$\max\{F_r(0, 0), F_r(0, 1)\}.$$

This is exactly the maximum of $\Phi_{\alpha,\beta,\gamma}(T, U)$ over all subsets $U \subseteq V(T)$.

Each vertex contributes through exactly one evaluation of $\varphi_{\alpha,\beta,\gamma}$, namely when the state at that vertex is computed, so no vertex is double counted. Substituting the six triples listed above gives the six parameters.

To construct an optimal set, store one maximizing choice whenever a maximum is taken in the computation of H_{s+1} or $F_v(a, b)$. After choosing a root state attaining $\max\{F_r(0, 0), F_r(0, 1)\}$, recursively follow the stored choices to decide the membership of every child. The correctness of the recurrence shows that the resulting set U attains the computed value.

For each vertex v , each $b \in \{0, 1\}$, and each child $u \in C(v)$, the transition updates three capped states and two choices of c . Thus the work at v is bounded by a constant times $|C(v)|$. Since

$$\sum_{v \in V(T)} |C(v)| = |V(T)| - 1,$$

the running time is $O(|V(T)|)$ for each fixed triple. Running the algorithm for the six fixed triples is still linear. The stored values and choices require linear memory. $\square$

4 NP-completeness on general graphs

For arbitrary graphs, we consider the six associated decision problems. Given a graph G , an integer t , and a certificate $U \subseteq V(G)$, one computes $p_U(v)$ for every vertex v and then determines whether v contributes to $E(U)$, $I(U)$ or $S(U)$. This verification is polynomial in $|V(G)| + |E(G)|$. Hence all six decision problems belong to NP.

We use the following standard NP-complete problems, see Karp [12] and Garey–Johnson [8].

INDEPENDENT SET. Given a graph H and an integer k , decide whether H has an independent set of size at least k .

MAX-CUT. Given a graph H and an integer k , decide whether H has a set $A \subseteq V(H)$ such that at least k edges of H have exactly one endpoint in A .

4.1 The parameters IPN and $ISPN$

Let H be a graph. Construct a graph $B(H)$ as follows. For each vertex $v \in V(H)$, replace v by two adjacent vertices v^0, v^1 . For every edge $uv \in E(H)$, join each of u^0, u^1 to each of v^0, v^1 . Thus each original vertex becomes a copy of K_2 , and adjacent original vertices are replaced by completely joined copies of K_2 .

Lemma 4.1. *For every graph H ,*

$$IPN(B(H)) = ISPN(B(H)) = 2\alpha(H),$$

where $\alpha(H)$ denotes the independence number of H .

Proof. Let A be an independent set in H . Put

$$U_A = \{v^0, v^1 : v \in A\}.$$

Since A is independent, $B(H)[U_A]$ is a disjoint union of copies of K_2 . Hence every vertex of U_A has exactly one neighbor in U_A . Thus $I(U_A) = 2|A|$, and so $IPN(B(H)) \geq 2\alpha(H)$ and $ISPN(B(H)) \geq 2\alpha(H)$.

We prove the reverse inequalities. Let $C \subseteq V(B(H))$ be any set satisfying $\Delta(B(H)[C]) \leq 1$. For each $v \in V(H)$, define

$$m_v = |C \cap \{v^0, v^1\}| \in \{0, 1, 2\},$$

and put $R = \{v \in V(H) : m_v > 0\}$. If $m_v = 2$, then v has no neighbor in $H[R]$, because any selected vertex in a neighboring copy would give each of v^0, v^1 at least two neighbors inside C . If $m_v = 1$, then the unique vertex in $C \cap \{v^0, v^1\}$ has at most one neighbor in C , and therefore

$$\sum_{u \in N_H(v) \cap R} m_u \leq 1.$$

Consequently every component of $H[R]$ is either an isolated vertex, or a single edge whose two endpoints both have m -value 1. From each component choose one vertex. These chosen vertices form an independent set in H , and each component contributes at most two vertices to C . Hence $|C| \leq 2\alpha(H)$.

For $ISPN$, given any $U \subseteq V(B(H))$, the vertices counted by $I(U) + S(U)$ are precisely those vertices of U having degree at most 1 in $B(H)[U]$. Let C be this counted set. Then $\Delta(B(H)[C]) \leq 1$, and therefore $I(U) + S(U) = |C| \leq 2\alpha(H)$. Hence $ISPN(B(H)) \leq 2\alpha(H)$.

For IPN , given any $U \subseteq V(B(H))$, the vertices counted by $I(U)$ are selected vertices having degree exactly 1 in $B(H)[U]$. Let C be this counted set. Then $\Delta(B(H)[C]) \leq 1$, and therefore $I(U) = |C| \leq 2\alpha(H)$. Hence $IPN(B(H)) \leq 2\alpha(H)$.

Together with the lower bounds, this proves the lemma. $\square$

Theorem 4.2. *The decision problems for IPN and $ISPN$ are NP-complete. Consequently, exactly computing $IPN(G)$ or $ISPN(G)$ on arbitrary graphs is NP-hard.*

Proof. Both problems belong to NP. Given an instance (H, k) of INDEPENDENT SET, construct $B(H)$ and set $t = 2k$. By Lemma 4.1,

$$IPN(B(H)) \geq 2k \iff \alpha(H) \geq k,$$

and the same equivalence holds for $ISPN$. The construction is polynomial, so the two decision problems are NP-hard. Hence they are NP-complete. A polynomial-time exact algorithm for either optimization problem would solve the corresponding decision problem by comparing the computed value with t , so the exact optimization problems are NP-hard. $\square$

4.2 The parameters EPN , $EIPN$, $ESPN$ and $EISPN$

We now prove NP-completeness for EPN , $EIPN$, $ESPN$ and $EISPN$ by reductions from MAX-CUT. The reductions use gadgets of constant size attached to the endpoints of each edge.

Let Q be a graph with two distinguished terminals x, y and some internal vertices. For a parameter $P \in \{EPN, EIPN, ESPN, EISPN\}$, define $f_P(s)$ to be the maximum number of internal vertices of Q that can be counted for P , over all choices of the internal vertices in U , subject to the condition that exactly s of the two terminals x, y belong to U . The terminals themselves are not counted in $f_P(s)$.

Lemma 4.3. *For each $P \in \{EPN, EIPN, ESPN, EISPN\}$ there is a constant-size gadget with terminals x, y whose values are as follows:*

Table 2: Gadget values.

P	$f_P(0)$	$f_P(1)$	$f_P(2)$
EPN	0	1	0
$EIPN$	0	1	0
$ESPN$	3	4	3
$EISPN$	4	5	4

Proof. For EPN and $EIPN$, take a single internal vertex q adjacent to both terminals x, y . If exactly one terminal is selected, then q is counted by leaving it outside U ; for $EIPN$, it could also be selected and counted internally. If no terminal or both terminals are selected, then q has respectively zero or two selected terminal-neighbors, and it cannot contribute to EPN or $EIPN$. Thus the first two rows follow.

For $ESPN$, use four internal vertices a, b, c, d with edges ac, ad, bc, bd , and with d adjacent to both terminals x, y , while a, b, c are adjacent to no terminal. If no terminal is selected, taking $U \cap V(Q) = \{a\}$ counts a as self private and counts c, d as external private. Hence $f_{ESPN}(0) \geq 3$. To see that all four internal vertices cannot be counted when no terminal is selected, observe that every selected counted vertex must be self private, so $U \cap \{a, b, c, d\}$ must be independent if all selected internal vertices are counted. In the 4-cycle with bipartition $\{a, b\}, \{c, d\}$, an independent set of size one leaves the non-neighbor of the selected vertex outside U with no selected neighbor, while an independent set of size two must be one side of the bipartition and then each vertex on the other side has two selected neighbors. Thus no choice counts all four internal vertices, and $f_{ESPN}(0) = 3$.

If exactly one terminal is selected, take $U \cap V(Q) = \{c\}$. Then c is self private, a and b are external private through c , and d is external private through the selected terminal. Hence all four internal vertices are counted, so $f_{ESPN}(1) = 4$. If both terminals are selected, then d cannot be counted, because it has at least two selected terminal-neighbors whether or not $d \in U$. Taking $U \cap V(Q) = \{c\}$ counts a, b, c , so $f_{ESPN}(2) = 3$.

For $EISP_N$, use five internal vertices a, b, c, d, e with edges ab, ac, bc, bd, ae , and with e adjacent to both terminals x, y , while a, b, c, d are adjacent to no terminal. Thus a, b, c span a triangle, d is a leaf adjacent to b , and e is adjacent to a and to the two terminals.

If no terminal is selected, taking $U \cap V(Q) = \{b\}$ counts b as self private and counts a, c, d as external private. Hence $f_{EISP_N}(0) \geq 4$. We prove that all five internal vertices cannot be counted when no terminal is selected. Suppose, for a contradiction, that all five are counted. Since c is adjacent to both a and b , the vertices a and b cannot both belong to U : if they did, then c would have two selected neighbors and would not be counted, regardless of whether $c \in U$.

If neither a nor b belongs to U , then c must belong to U in order to be counted, because otherwise c would have no selected neighbor. Also d must belong to U , since its only neighbor is $b \notin U$, and e must belong to U , since no terminal is selected and its only non-terminal neighbor is $a \notin U$. But then $a \notin U$ has two selected neighbors, namely c and e , and so a is not counted, a contradiction.

Hence exactly one of a, b belongs to U . If $a \in U$ and $b \notin U$, then b is outside U and must have exactly one selected neighbor among a, c, d . Since a is already selected, this forces $c, d \notin U$. But then $d \notin U$ has no selected neighbor, because its only neighbor is $b \notin U$, and so d is not counted, a contradiction. The case $b \in U$ and $a \notin U$ is symmetric in the following explicit sense: since a is outside U and must have exactly one selected neighbor among b, c, e , the selected vertex b forces $c, e \notin U$; then $e \notin U$ has no selected neighbor, because no terminal is selected and $a \notin U$, so e is not counted. This contradiction proves that at most four internal vertices can be counted, and therefore $f_{EISP_N}(0) = 4$.

If exactly one terminal is selected, take $U \cap V(Q) = \{b\}$. Then b is self private, the vertices a, c, d are external private through b , and e is external private through the selected terminal. Hence all five internal vertices are counted, so $f_{EISP_N}(1) = 5$. If both terminals are selected, then e cannot be counted, because it has the two selected terminals as neighbors whether or not $e \in U$. Taking again $U \cap V(Q) = \{b\}$ counts a, b, c, d , and hence $f_{EISP_N}(2) = 4$. This proves the final row. $\square$

Fix $P \in \{EPN, EIPN, ESPN, EISP_N\}$. Let B_P be the common value of the first and third columns in the row for P in Lemma 4.3. Thus

$$B_{EPN} = B_{EIPN} = 0, \quad B_{ESPN} = 3, \quad B_{EISP_N} = 4,$$

and the gadget has value $B_P + 1$ precisely when exactly one terminal is selected.

Given an instance (H, k) of MAX-CUT, let $n_0 = |V(H)|$, $m_0 = |E(H)|$ and $R = n_0 + 1$. Construct a graph $G_P(H)$ as follows. Begin with the vertex set $V(H)$, made independent. For each edge $xy \in E(H)$, attach R disjoint copies of the gadget for P , using x and y as the terminals of each copy. The internal vertices of different gadget copies are disjoint and have no edges between them. Set $T_P = R(B_P m_0 + k)$.

Lemma 4.4. *For each $P \in \{EPN, EIPN, ESPN, EISP_N\}$,*

$$P(G_P(H)) \geq T_P \iff H \text{ has a cut of size at least } k.$$

Proof. Suppose first that $A \subseteq V(H)$ is a cut of size at least k . Put the vertices of A into U . For each gadget copy attached to an edge xy , choose an internal subset attaining the value given by Lemma 4.3, according to whether 0, 1 or 2 of x, y lie in A . Each non-crossing edge contributes at least B_P from each of its R gadget copies, and each crossing edge contributes $B_P + 1$ from each of its R gadget copies. Thus the total contribution from internal gadget vertices is at least

$$R(B_P m_0 + k) = T_P.$$

Any contribution from the terminal vertices in $V(H)$ can only increase the value. Hence $P(G_P(H)) \geq T_P$.

Conversely, suppose that $U \subseteq V(G_P(H))$ is arbitrary, and put $A = U \cap V(H)$. For each edge $xy \in E(H)$, Lemma 4.3 implies that each gadget copy attached to xy contributes at most $B_P + 1$ internal vertices if exactly one of x, y lies in A , and at most B_P internal vertices otherwise. Therefore the total contribution from all internal gadget vertices is at most

$$R(B_P m_0 + \text{cut}_H(A)),$$

where $\text{cut}_H(A)$ is the number of edges of H with exactly one endpoint in A .

The vertices belonging to the original set $V(H)$ contribute at most n_0 in total, since each vertex contributes at most once. Hence

$$P(G_P(H)) \leq R(B_P m_0 + \text{cut}_H(A)) + n_0.$$

If H has no cut of size at least k , then $\text{cut}_H(A) \leq k - 1$ for every $A \subseteq V(H)$, and so

$$P(G_P(H)) \leq R(B_P m_0 + k - 1) + n_0 < R(B_P m_0 + k) = T_P,$$

because $R = n_0 + 1$. This proves the converse. $\square$

Theorem 4.5. *The decision problems for EPN, EIPN, ESPN and EISPN are NP-complete. Consequently, exactly computing any one of these four parameters on arbitrary graphs is NP-hard.*

Proof. Each problem belongs to NP. For each fixed parameter $P \in \{EPN, EIPN, ESPN, EISPN\}$, Lemma 4.4 gives a polynomial-time reduction from MAX-CUT to the decision problem $P(G) \geq t$. Hence each of the four decision problems is NP-hard, and therefore NP-complete. A polynomial-time exact algorithm for any corresponding optimization problem would solve its decision version, so the exact optimization problems are NP-hard. $\square$

Proof of Theorem 1.4. Combining Theorem 4.2 and Theorem 4.5 proves Theorem 1.4. $\square$

References

- [1] B. Bollobás and E. J. Cockayne, Graph-theoretic parameters concerning domination, independence, and irredundance, *J. Graph Theory* **3** (1979), 241–249. [2](#)

- [2] K. Cameron, Induced matchings, *Discrete Appl. Math.* **24** (1989), 97–102. [2](#)
- [3] E. J. Cockayne, Generalized irredundance in graphs: hereditary properties and Ramsey numbers, *J. Combin. Math. Combin. Comput.* **31** (1999), 15–31. [2](#)
- [4] E. J. Cockayne, S. T. Hedetniemi and D. J. Miller, Properties of hereditary hypergraphs and middle graphs, *Canad. Math. Bull.* **21** (1978), 461–468. [1](#)
- [5] A. M. Farley and N. Shachum, Senders in broadcast networks: open irredundancy in graphs, *Congr. Numer.* **38** (1983), 47–57. [2](#)
- [6] M. Fellows, G. Fricke, S. Hedetniemi and D. Jacobs, The private neighbor cube, *SIAM J. Discrete Math.* **7** (1994), 41–47. [2](#)
- [7] J. F. Fink and M. S. Jacobson, n -domination in graphs, in *Graph Theory with Applications to Algorithms and Computer Science*, Wiley, New York, 1985, 283–300. [2](#)
- [8] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, San Francisco, 1979. [6](#)
- [9] T. W. Haynes, S. T. Hedetniemi and M. A. Henning, *Domination in Graphs: Core Concepts*, Springer Monographs in Mathematics, Springer, Cham, 2023. [1](#)
- [10] S. M. Hedetniemi, S. T. Hedetniemi and D. P. Jacobs, Private domination: theory and algorithms, *Congr. Numer.* **79** (1990), 147–157. [2](#)
- [11] S. T. Hedetniemi and D. F. Rall, On maximizing private neighbors in graphs, arXiv:2511.07248, 2025. [2](#)
- [12] R. M. Karp, Reducibility among combinatorial problems, in *Complexity of Computer Computations*, Plenum, New York, 1972, 85–103. [6](#)
- [13] C. M. Mynhardt and A. Roux, Irredundance, in *Structures of Domination in Graphs*, Developments in Mathematics, vol. 66, Springer, Cham, 2021, 135–181. [2](#)

A A dynamic programming algorithm on trees

For completeness, we record a dynamic programming algorithm for trees. The algorithm computes the maximum value of any prescribed linear combination of external, internal and self private neighbors, and also constructs an optimal set attaining this value.

Let $(\alpha, \beta, \gamma) \in \{0, 1\}^3$. Recall that, for $U \subseteq V(T)$,

$$\Phi_{\alpha, \beta, \gamma}(T, U) = \alpha E_T(U) + \beta I_T(U) + \gamma S_T(U).$$

The local contribution function is

$$\varphi_{\alpha,\beta,\gamma}(b,t) = \begin{cases} \alpha, & b = 0 \text{ and } t = 1, \\ \beta, & b = 1 \text{ and } t = 1, \\ \gamma, & b = 1 \text{ and } t = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Here b records whether the vertex itself belongs to U , while t records the number of its neighbors that belong to U . In the algorithm below, $-\infty$ denotes a value smaller than every feasible objective value.

Algorithm 1: Computing an optimal set for $\Phi_{\alpha,\beta,\gamma}$ on a tree

Input: A non-empty tree T and a triple $(\alpha, \beta, \gamma) \in \{0, 1\}^3$.

Output: The value $\max_{U \subseteq V(T)} \Phi_{\alpha,\beta,\gamma}(T, U)$ and an optimal set U attaining it.

```

1: Choose a root  $r$  of  $T$  and list the vertices in postorder.
2: for all vertices  $v$  in postorder do
3:   List the children of  $v$  as  $u_1, \dots, u_m$ .
4:   for  $b \in \{0, 1\}$  do
5:     Set  $H_0(0) \leftarrow 0$  and  $H_0(1), H_0(2) \leftarrow -\infty$ .
6:     for  $s = 0, \dots, m - 1$  do
7:       Set  $H_{s+1}(0), H_{s+1}(1), H_{s+1}(2) \leftarrow -\infty$ .
8:       for  $q \in \{0, 1, 2\}$  do
9:         for  $c \in \{0, 1\}$  do
10:           $q' \leftarrow \min\{2, q + c\}$ .
11:          if  $H_s(q) + F_{u_{s+1}}(b, c) > H_{s+1}(q')$  then
12:             $H_{s+1}(q') \leftarrow H_s(q) + F_{u_{s+1}}(b, c)$ .
13:             $\text{PredG}[v, b, s + 1, q'] \leftarrow (q, c)$ .
14:          end if
15:        end for
16:      end for
17:    end for
18:    for  $q \in \{0, 1, 2\}$  do
19:       $G_v^b(q) \leftarrow H_m(q)$ .
20:    end for
21:  end for
22:  for  $a \in \{0, 1\}$  do
23:    for  $b \in \{0, 1\}$  do
24:      Choose  $q^* \in \{0, 1, 2\}$  maximizing  $G_v^b(q) + \varphi_{\alpha,\beta,\gamma}(b, a + q)$ .
25:       $F_v(a, b) \leftarrow G_v^b(q^*) + \varphi_{\alpha,\beta,\gamma}(b, a + q^*)$ .
26:       $\text{PredF}[v, a, b] \leftarrow q^*$ .

```

```

27:         end for
28:     end for
29: end for
30: Choose  $b_r \in \{0, 1\}$  maximizing  $F_r(0, b)$ .
31: Initialize  $U \leftarrow \emptyset$ .
32: Call RECOVER( $r, 0, b_r$ ).
33: return  $F_r(0, b_r)$  and  $U$ .

```

Algorithm 2 RECOVER(v, a, b)

Input: A state (v, a, b) , the arrays PredF and PredG, and the current set U .

Output: The membership decisions in the subtree T_v are added to U .

```

1: if  $b = 1$  then
2:     Add  $v$  to  $U$ .
3: end if
4: List the children of  $v$  as  $u_1, \dots, u_m$ .
5:  $q_m \leftarrow \text{PredF}[v, a, b]$ .
6: for  $s = m$  down to 1 do
7:      $(q_{s-1}, c[s]) \leftarrow \text{PredG}[v, b, s, q_s]$ .
8: end for
9: for  $s = 1, \dots, m$  do
10:    Call RECOVER( $u_s, b, c[s]$ ).
11: end for

```

The correctness of the algorithm is exactly the correctness of the recurrence proved in Section 3. To construct an optimal set, we store one maximizing choice whenever a maximum is taken: for each state $F_v(a, b)$ we store a maximizing value of q , and during the computation of G_v^b we store the corresponding predecessor choices for the child-subtrees. Starting from a root state attaining $\max\{F_r(0, 0), F_r(0, 1)\}$, standard backtracking through these stored choices determines an optimal set U .

The running time is linear, since for each vertex v , each $b \in \{0, 1\}$, and each child of v , only constantly many states are updated. Thus the total work is

$$O\left(\sum_{v \in V(T)} |C(v)|\right) = O(|V(T)|).$$

The number of stored values and choices is also linear in $|V(T)|$. Hence, for each fixed triple (α, β, γ) , the algorithm computes the optimum and constructs an optimal set in linear time and linear space. Applying it to the six triples corresponding to IPN , EPN , $ESPN$, $EIPN$, $ISPN$ and $EISPN$ still gives linear time.